Modelado de datos con estructuras (struct) y punteros en C++

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0 — Baseline aprobado

Modelado de datos con estructuras (struct) y punteros en C++

Introducción

Los programas reales manejan entidades compuestas (sensores, módulos de hardware, usuarios) cuyos datos son de distintos tipos. En C++ se modelan con estructuras (`struct`), que agrupan esas variables bajo una misma entidad lógica en bloques contiguos de memoria. Si además se pasa la estructura a una función por dirección, se modifica el original sin duplicarlo.

La actividad modela una `EntidadProyecto` (id, nombre y métrica), carga sus datos mediante un puntero con el operador flecha (`->`) y muestra los datos verificados junto con su dirección hexadecimal. Se documenta como base para estructurar las variables del Proyecto Integrador Anual de la Expotécnica. Integrante: Alexis Maier (trabajo individual: código, capturas, manual e informe).

Desarrollo

Estructura y función de carga

```
struct EntidadProyecto {
   int id;
   char nombre[50];
   float metrica;
};

void cargarDatos(EntidadProyecto* ptr);   // pasaje por dirección
```

En `main` se declara `miEntidad` inicializada de forma segura y se invoca `cargarDatos(&miEntidad)`. Dentro de la función, `ptr->id`, `ptr->nombre` y `ptr->metrica` acceden directamente a los miembros del original. Entre el `cin >> ptr->id` y el `cin.getline` se usa `cin.ignore()` para descartar el Enter que quedó en el búfer.

Compilación y ejecución

Se compiló con `g++ src/main.cpp -o src/sistemaproyecto.exe`. Se cargó como ejemplo un sensor de temperatura (ID 101, «Sensor de temperatura DHT22», métrica 23,5).

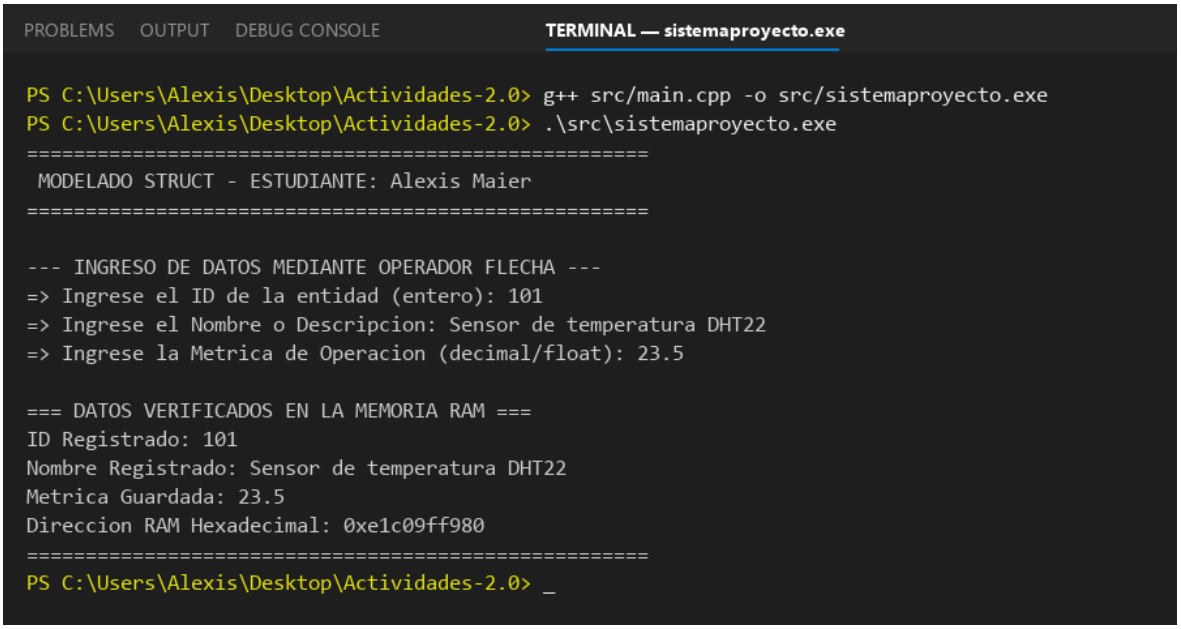


Figura 1. Carga de datos mediante el operador flecha y verificación en memoria.

Cuestionario técnico de control

1. Heterogeneidad: ¿Qué ventaja ofrece agrupar los datos de un sensor o dispositivo en un struct en lugar de declarar muchas variables sueltas?

El struct reúne en una sola entidad lógica datos de distinto tipo (un entero, una cadena y un decimal), de modo que el sensor se maneja como una unidad: se pasa a una función con un solo parámetro, se copia o se guarda en arreglos de sensores y el código se lee como el problema real. Con variables sueltas habría que mantener sincronizadas muchas variables y pasar cada una por separado, lo que aumenta los errores.

2. Operador flecha: ¿Por qué el compilador de C++ no permite usar la notación de punto (ptr.id) cuando la variable es de tipo puntero?

Porque `ptr` no es la estructura sino una variable que contiene su dirección; el operador punto accede a miembros de un objeto, no de una dirección. Para llegar al campo hay que desreferenciar primero, `(*ptr).id`, y `->` es la sintaxis abreviada de esa operación.

3. Eficiencia en RAM: ¿Por qué el pasaje por referencia mediante punteros optimiza la memoria procesada frente al pasaje por copia?

Con el pasaje por copia, cada llamada duplica toda la estructura en el stack; con punteros solo se envía la dirección (8 bytes en 64 bits), sin importar el tamaño del struct. Además la función trabaja sobre el original, así que no hace falta devolver ni reasignar los datos.

4. Buffer de entrada: ¿Qué comportamiento erróneo ocurre en la terminal si se omite cin.ignore() antes de un getline?

Tras `cin >> ptr->id` el Enter queda en el buffer. Al llamar a `cin.getline`, este encuentra enseguida ese salto de línea, interpreta una línea vacía y termina sin esperar al usuario: el nombre queda vacío y el programa pasa directo a pedir la métrica.

5. Aplicación al Proyecto Integrador: ¿Qué entidad o componente hardware de su P.I.A. representaron con esta estructura y qué variables componen sus miembros?

Se representó un sensor del prototipo (ejemplo de la ejecución: un sensor de temperatura DHT22). Sus miembros son `id` (identificador del sensor en el sistema), `nombre` (descripción del componente) y `metrica` (última lectura, en °C). Para el P.I.A. real se pueden agregar campos como unidad de medida, estado o fecha de la lectura.

Control de versiones

El historial de desarrollo se versiona con Git en la rama `modelado-datos-struct`, creada a partir de `main`, con commits descriptivos. El manual del programador se encuentra en `docs/manuales/manual_programador_v1.0.0.pdf` (y `.md`).

Conclusión

El `struct` permite tratar un dispositivo como una unidad y el pasaje por puntero con `->` evita copias innecesarias en la RAM. Se comprobó además la importancia de limpiar el búfer con `cin.ignore()` antes de leer cadenas. Esta estructura es una base directa para representar los sensores o módulos del Proyecto Integrador Anual.

Referencias

cplusplus.com. (s. f.). *Data structures*. https://cplusplus.com/doc/tutorial/structures/

cplusplus.com. (s. f.). *Pointers*. https://cplusplus.com/doc/tutorial/pointers/

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Mansilla Muñoz, Y. E. (2026). *Actividad 5: Modelado de datos con estructuras (struct) y punteros* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».